

Unit 7: Decidability

Decidable Problems

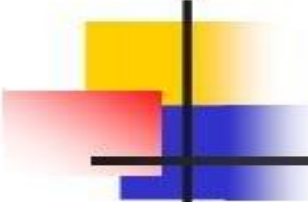
- A problem is *decidable* if there is an algorithm to answer it.
 - **Recall:** An “algorithm,” formally, is a TM that halts on all inputs, accepted or not.
 - Put another way, “decidable problem” = “recursive language.”
- Otherwise, the problem is *undecidable*.

Problems

- Informally, a “problem” is a yes/no question about an infinite set of possible *instances*.
- **Example:** “Does graph G have a *Hamilton cycle* (cycle that touches each node exactly once)?
 - Each undirected graph is an instance of the “Hamilton-cycle problem.”

Example: A Problem About Turing Machines

- We can think of the language L_d as a problem.
- “Does this TM not accept its own code?”
- **Aside:** We could also think of it as a problem about binary strings.
 - Do you see how to phrase it?



Undecidable problems

- A decision problem (or language) is **decidable** if there is a Turing Machine that solves that problem
 - Leaves a YES on the tape when the string is acceptable, leaves a NO when it is not
- A problem (or language) is undecidable if there is no Turing Machine that solves (or decides) that problem
- The terms solvable/unsolvable are sometimes used instead of decidable/undecidable

Examples: Undecidable Problems

- Can a particular line of code in a program ever be executed?
- Is a given context-free grammar ambiguous?
- Do two given CFG's generate the same language?

The Universal Language

- An example of a recursively enumerable, but not recursive language is the language L_u of a *universal Turing machine*.
- That is, the UTM takes as input the code for some TM M and some binary string w and accepts if and only if M accepts w .

- **Universal Turing Machine:** A Turing machine can also compute everything that a real computer can compute. For example, a Turing machine can simulate any function used in a programming language.
- **Recursive Language:** A recursive language is a formal language for which there exists a Turing machine that, when presented with any finite input string, halts and accepts if the string is in the language, and halts and rejects otherwise.

It is a mathematical model of computation it manipulates symbols on the tape according to the rules defined	Universal Turing Machine is like a single Turing Machine that has a solution to all problem that is computable
A program can be compared to a Turing Machine Turing machine's temporary storage is tape. The infinite cells of the Turing machine can contain input symbols and blanks. <u>Turing machines</u> help us understand the fundamental limitations of mechanical computation power A Turing machine is a formal model of a computer with a fixed program	Programmable Turing Machine is called Universal Turing Machine Universal Turing Machine contains Turing Machine description as input along with an input string, runs the Turing Machine on the input and returns the result. Although developed for theoretical reasons, it helped in the development of stored program computers Universal Turing Machine provides a solution to problems that are computable
It does not minimize the space complexity Transition function which Turing Machine performs is defined as: $\delta \times T \rightarrow Q \times T \times \{L, R\}$, where δ is the transition function In the <u>set theory</u> point of view, all Turing machines form a set of the all the device that accepts type 0 grammar	It minimizes <u>space complexity</u> The transition function is $Q \times T \rightarrow Q \times T \times \{L, R\}$, where Q is a finite set of states, T is the tape of the alphabet Universal Turing Machine is a subset of all the Turing Machines

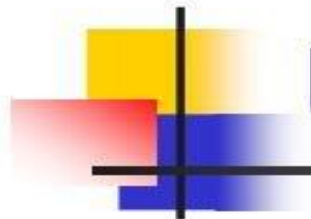
Diagonalization languages - Table of Acceptance

		String $j \longrightarrow$						
		1	2	3	4	5	6	...
TM $i \downarrow$	1							
	2							
	3							
	4							
	5							
	6							
	.							
	.							
	.							
	.							

$x = 0$ means
the i -th TM does
not accept the
 j -th string; 1
means it does.

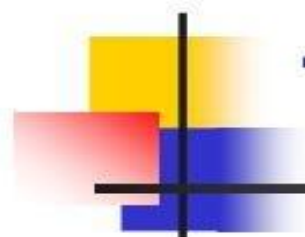
Church-Turing Thesis

- A Turing machine that halts on all inputs is the precise formal notion corresponding to the intuitive notion of an algorithm
- An “algorithm” means a precisely defined set of instructions
- This thesis cannot be formally proven
- Consequence of Church-Turing thesis: If a problem cannot be solved by a Turing machine then it cannot be solved by a human using a precisely defined sequence of instructions, either
- Knowing a problem is undecidable helps avoid wasted work
- Can extend Church-Turing thesis to semidecidability (e.g. theorem proving)



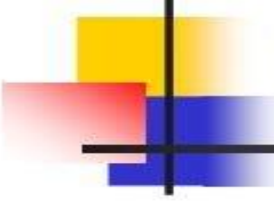
Using a TM to solve a problem

- Recall the two possible interpretations of “solving” a problem through Turing Machines
- Device a TM so that acceptable (input) strings in the corresponding language are those that end up in a final state
 - When not acceptable, we don't need to care if the TM halts
- Device a machine so that it leaves YES on the tape when the string is acceptable, NO if not
 - Note: steeper requirement than the first option



The halting problem

- The Halting Problem (HP):
Given a TM P and input i , does P halt on i ?
- HP is undecidable
- That is, there is no program (turing machine) that solves HP
- If there was such a program
 - Its input will have two portions, P and i
 - It outputs either a YES or a NO depending on whether P halts on input i



HP is undecidable

- Proof by contradiction: suppose HP is decidable
 - Plan: arrive at a contradiction
- If HP is decidable, then there exists a program A that solves HP:

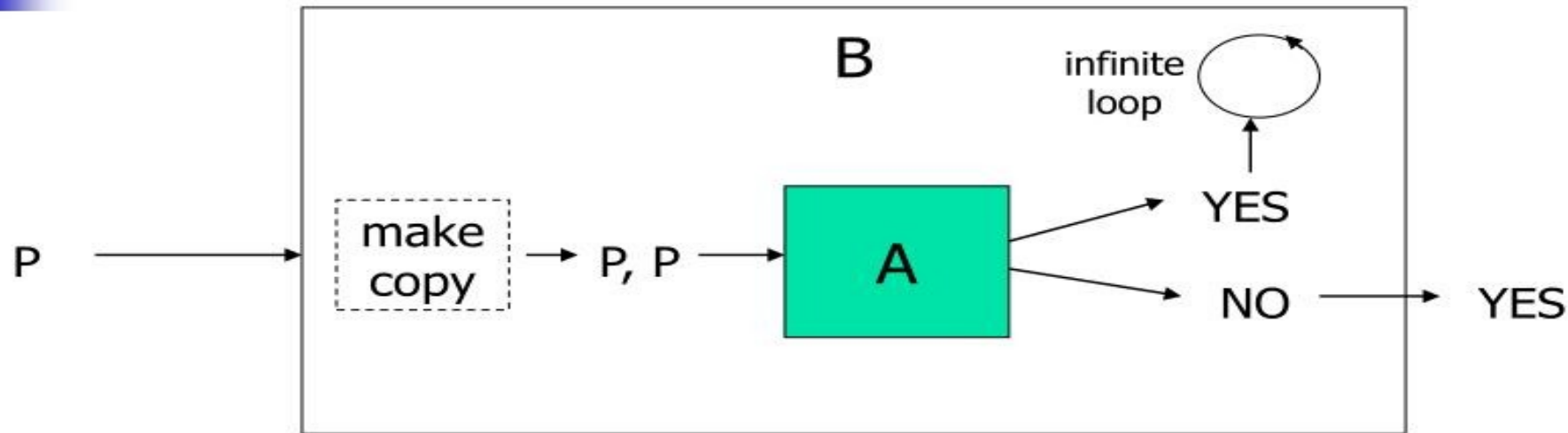




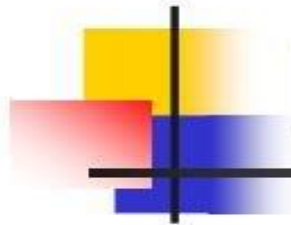
HP is undecidable

- Create a program B based on A as follows:
 - B takes in a program P
 - In B, P is duplicated so that there are now two portions on the input tape
 - Feed this new input into A
 - When it is about to print NO, print YES instead
 - When it is about to print YES, send the program to an infinite loop

HP is undecidable

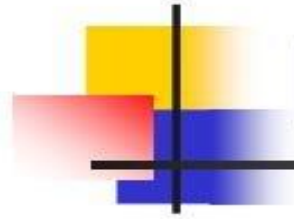


Program B takes a program P as input, prints a YES if P **does not halt** on input P , but goes into an infinite loop if P **halts** on input P

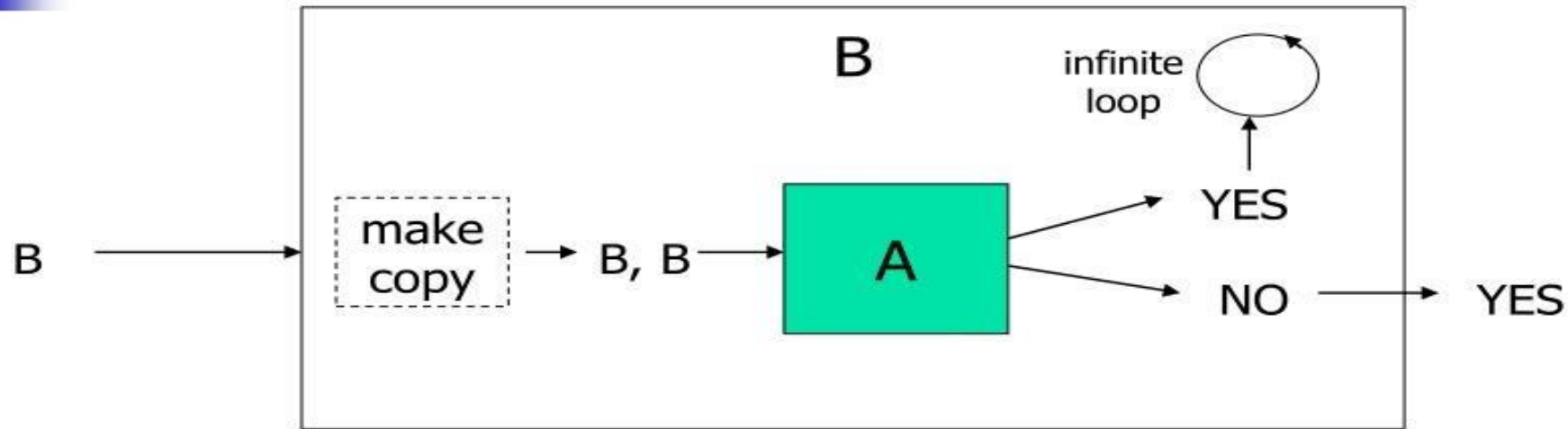


HP is undecidable

- Consider feeding program B to itself
- Consequence (two possibilities)
 - It prints a YES
 - B halts on input B
if B does not halt on input B $\rightarrow$ a contradiction
 - It goes to an infinite loop
 - B does not halt on input B
if B halts on input B $\rightarrow$ a contradiction
- Therefore the supposition cannot hold, and HP is undecidable

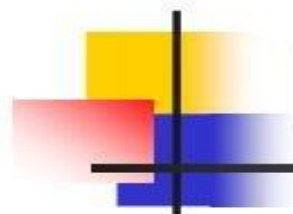


Feeding program B to itself



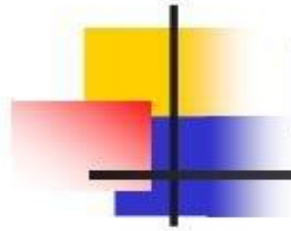
B halts on input B (prints a YES, see outer box) if
B does not halt on input B (A should yield a NO, see inner box)

B does not halt on input B (infinite loop, see outer box) if
B halts on input B (A should yield a YES, see inner box)

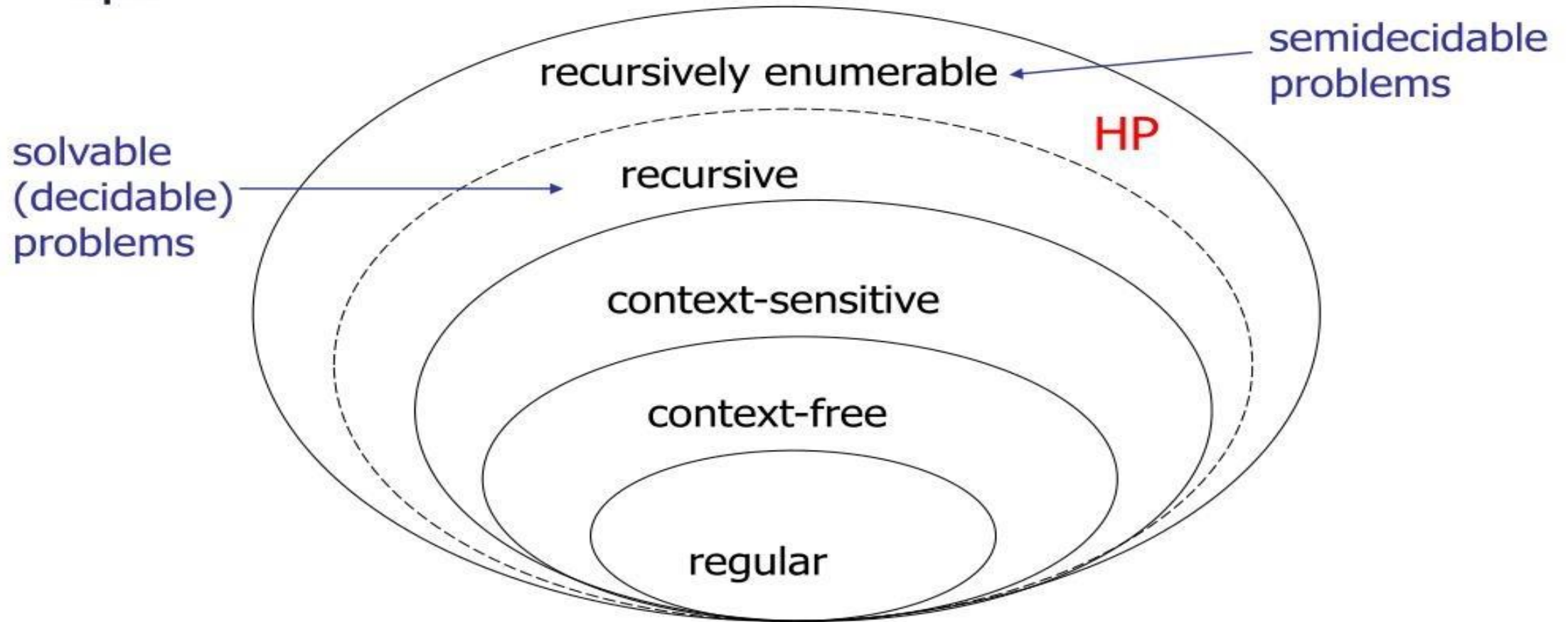


Notes and Conclusions

- There are problems such as HP that cannot be solved
- Actually, HP is **semidecidable**, that is if all we need is print YES when P on i halts, but not worry about printing NO if otherwise, a TM machine exists for the halting problem
 - Just simulate P on i, print YES (or go to a final state) when the simulation stops
 - This means that HP is not recursive but it is recursively enumerable



HP and the Chomsky hierarchy



Comparison	Recursive Language	Recursively enumerable language
Also Known as	Turing decidable languages	Turing recognizable languages
Definition	In Recursive Languages, the Turing machine accepts all valid strings that are part of the language and rejects all the strings that are not part of a given language and halt.	In Recursively enumerable languages, the Turing machine accepts all valid strings that are part of the language and rejects all the strings that are not part of the given language but do not halt and starts an infinite loop.
Loop	Finite Loop	Infinite loops of machine are possible
Halting	Halting Turing Machine	Non Halting Turing Machine
Closed under	Closed under all except homomorphism, substitution, GSM mapping, and rational transduction	Closed under all except set difference, and complementation.
	context-sensitive language	RE languages or type-0 language
Example	Language of even-length strings over {a, b}: $L = \{ w \in \{a, b\}^* \mid w \text{ is even} \}$ - A Turing machine can be constructed to count the length of	Halting problem language: $L = \{ \langle M, w \rangle \mid \text{Turing machine } M \text{ halts on input } w \}$ - There exists a Turing machine that enumerates pairs $\langle M, w \rangle$ and simulates M on input w. It accepts $\langle M, w \rangle$ if M halts on w.

Rice Theorem

- Rice theorem states that any non-trivial semantic property of a language which is recognized by a Turing machine is undecidable. A property P , is the language of all Turing machines that satisfy that property.

Formal Definition

- If P is a non-trivial property, and the language holding the property, L_p , is recognized by Turing machine M , then $L_p = \{ \langle M \rangle \mid L(M) \in P \}$ is undecidable.

Rice Theorem - Description and Properties

- Property of languages, P , is simply a set of languages. If any language belongs to P ($L \in P$), it is said that L satisfies the property P .
- A property is called to be trivial if either it is not satisfied by any recursively enumerable languages, or if it is satisfied by all recursively enumerable languages.
- A non-trivial property is satisfied by some recursively enumerable languages and are not satisfied by others. Formally speaking, in a non-trivial property, where $L \in P$, both the following properties hold:
 - **Property 1** – There exists Turing Machines, $M1$ and $M2$ that recognize the same language, i.e. either ($\langle M1 \rangle, \langle M2 \rangle \in L$) or ($\langle M1 \rangle, \langle M2 \rangle \notin L$)
 - **Property 2** – There exists Turing Machines $M1$ and $M2$, where $M1$ recognizes the language while $M2$ does not, i.e. $\langle M1 \rangle \in L$ and $\langle M2 \rangle \notin L$

Types Of Properties

❖ Trivial Properties

The constant map $RE \rightarrow \{T, F\}$ taking all $L \in RE$ to T

The constant map $RE \rightarrow \{T, F\}$ taking all $L \in RE$ to F

❖ *Non Trivial Properties*

Any other Properties it is called Non Trivial Properties it means

The constant map $RE \rightarrow \{T, F\}$ taking all $L \notin RE$ to T or F

Examples of Various Properties

- ❖ Finiteness property: Any member of $\{M \mid L(M) \text{ is finite}\}$
- ❖ Regularity property: Any member of $\{M \mid L(M) \text{ is regular}\}$
- ❖ Context-free property: Any member of $\{M \mid L(M) \text{ is context free}\}$
- ❖ Acceptance of a string: Any member of $\{M \mid 01011000 \in L(M)\}$
- ❖ Full-ness property: Any member of $\{M \mid L(M) = \Sigma^*\}$

Unit 7: Basic Introduction to Complexity

P, NP, NP-hard & NP-Completeness

P Class: The P in the P class stands for Polynomial Time. It is the collection of decision problems (problems with a “yes” or “no” answer) that can be solved by a deterministic machine in polynomial time.

Example of P class Problem

1. Calculating the greatest common divisor.
2. Finding a maximum matching.
3. Decision versions of linear programming.

NP Class: The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a non-deterministic machine in polynomial time.

Example of P class Problem

1. Boolean Satisfiability Problem (SAT).
2. Hamiltonian Path Problem.
3. Graph coloring.

NP-Hard Class: An NP-hard problem is at least as hard as the hardest problem in NP and it is the class of the problems such that every problem in NP reduces to NP-hard.

Example of NP-Hard class Problem

1. Halting problem
2. Qualified Boolean formulas.
3. No Hamiltonian cycle

NP Complete Class: A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

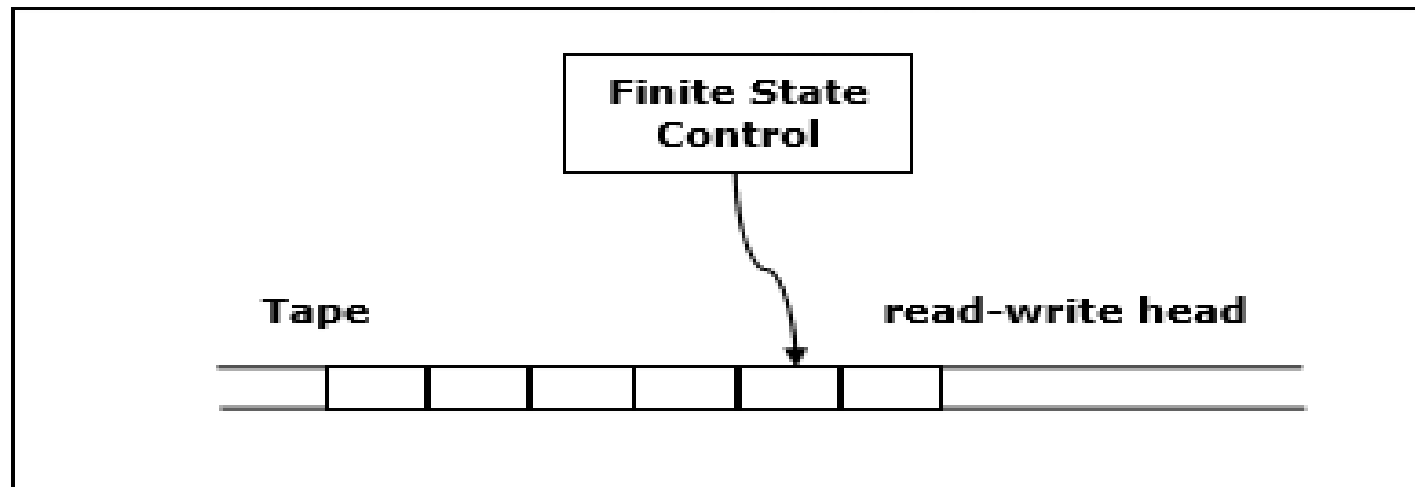
Example of NP-Complete class Problem

1. Decision version of 0/1 Knapsack.
2. Hamiltonian Cycle.
3. Vertex cover.

Time complexity of Deterministic & Non-Deterministic Turing Machine

Deterministic Turing Machine

- One of these models is deterministic one-tape Turing machine. This machine consists of a finite state control, a read-write head and a two-way tape with infinite sequence.
- Following is the schematic diagram of a deterministic one-tape Turing machine.



Deterministic Turing Machine

A program for a deterministic Turing machine specifies the following information –

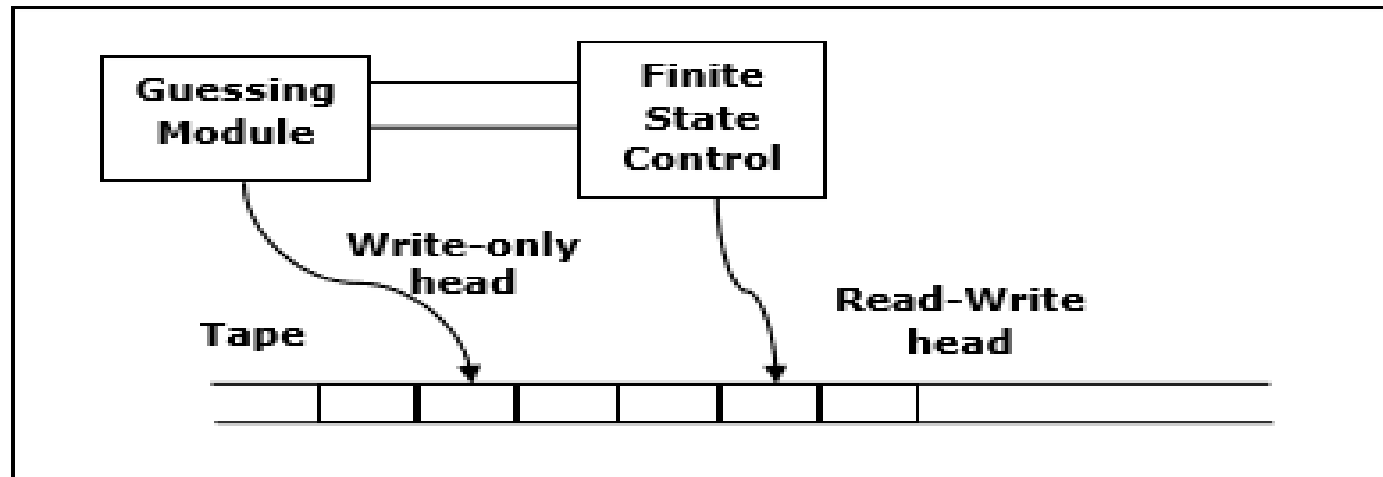
- A finite set of tape symbols (input symbols and a blank symbol)
- A finite set of states
- A transition function

In algorithmic analysis, if a problem is solvable in polynomial time by a deterministic one tape Turing machine, the problem belongs to **P class**.

Nondeterministic Turing Machine

To solve the computational problem, another model is the Nondeterministic Turing Machine (NDTM). The structure of NDTM is similar to DTM, however here we have one additional module known as the guessing module, which is associated with one write-only head.

Following is the schematic diagram.



If the problem is solvable in polynomial time by a non-deterministic Turing machine, the problem belongs to **NP class**.